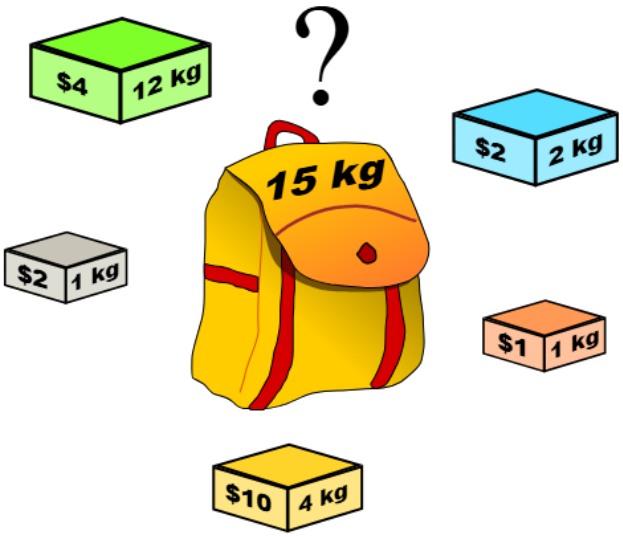
Tugas Besar 1

Penerapan Algoritma *Local Search* untuk Optimasi Pemilihan dan Penataan Paket dalam Peti Kemas

IF3170 Intelegensi Artifisial

Semester 1 Tahun 2026/2027



Disusun oleh:

Three Agents

Muhammad Aufar Rizqi Kusuma	(13524061)
Arina Azka	(13524049)
Michael James Liman	(13524106)

Laboratorium Intelegensi Buatan
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**
1.1 *Backstory* . . . 2
1.2 *Overview* Permasalahan . . . 2
2 Landasan Teori **3**
2.1 Algoritma Hill Climbing . . . 3
2.1.1 *Steepest Ascent Hill Climbing* . . . 3
2.1.2 *Hill Climbing with Sideways Move* . . . 3
2.1.3 *Stochastic Hill Climbing* . . . 3
2.1.4 *Random Restart Hill Climbing* . . . 4
2.2 Algoritma Simulated Annealing . . . 4
2.3 Genetic Algorithm . . . 4
3 Implementasi Algoritma **5**
3.1 Representasi State . . . 5
3.2 *Constraint* Permasalahan . . . 5
3.3 *Succesor Function* . . . 5
3.4 *Objective Function* . . . 5
3.5 Formulasi *Initial State* . . . 5
3.6 Implementasi Hill Climbing . . . 5
3.6.1 Pembangkitan *Neighbor* dan Heuristik . . . 5
3.6.2 Parameter dan Kondisi Berhenti . . . 5
3.6.3 Metrik yang Dicatat . . . 6
3.6.4 *Source Code* . . . 6
3.6.5 Pelaksana Eksperimen dan Visualisasi . . . 8
3.7 Implementasi Simullated Annealing . . . 9
3.8 Implementasi Genetic Algorithm . . . 9
4 Hasil Eksperimen dan Analisis **10**
4.1 Metodologi Eksperimen . . . 10
4.2 Hasil Eksperimen dan Analisis tiap Algoritma . . . 10
4.2.1 Hill Climbing . . . 10
4.3 Hasil Eksperimen dan Analisis tiap Algoritma . . . 10
4.3.1 Hill Climbing . . . 10
4.3.2 Simulated Annealing . . . 10
4.3.3 Genetic Algorithm . . . 10
4.4 Analisis Keseluruhan . . . 10
5 Kesimpulan dan Hasil **11**
5.1 Kesimpulan . . . 11
5.2 Saran . . . 11

Bab 1

Pendahuluan

1.1 *Backstory*

1.2 *Overview* Permasalahan

Bab 2

Landasan Teori

2.1 Algoritma Hill Climbing

Hill climbing adalah algoritma *local search* yang bekerja pada satu *state* saja. Algoritma ini mulai dari sebuah *state* awal, membangkitkan *neighbor* dari *state* saat ini, lalu berpindah ke *neighbor* yang nilai *objective function*-nya lebih baik. Pencarian berhenti ketika tidak ada lagi *neighbor* yang lebih baik. Karena hanya menyimpan satu *state* dan tidak menyimpan jalur pencarian, kebutuhan memorinya sangat kecil dan setiap iterasinya murah. Pada permasalahan ini *objective function* dimaksimalkan, sehingga algoritma bergerak "naik" menuju nilai *value* total paket yang lebih besar.

Kelemahan utama *hill climbing* adalah tidak adanya mekanisme untuk keluar dari daerah yang buruk. Ada tiga kondisi yang menyebabkan pencarian terhenti sebelum mencapai *global optimum*.

- *Local maximum*: *state* yang lebih baik dari semua *neighbor*-nya, tetapi bukan yang terbaik secara global.
- *Plateau*: daerah datar tempat semua *neighbor* bernilai sama, sehingga tidak ada arah yang jelas untuk naik.
- *Shoulder*: *plateau* yang di ujungnya masih terdapat jalan menaik, sehingga pencarian yang berhenti terlalu cepat melewatkannya.

Keempat varian berikut dibuat untuk mengurangi kelemahan tersebut dengan cara yang berbeda.

2.1.1 *Steepest Ascent Hill Climbing*

Pada setiap iterasi, algoritma mengevaluasi sekumpulan *neighbor* dan memilih *neighbor* dengan nilai terbaik. Algoritma berpindah ke *neighbor* tersebut hanya jika nilainya *strictly* lebih besar daripada nilai *state* saat ini; jika tidak, algoritma berhenti karena telah mencapai *local optimum*. Varian ini deterministik terhadap pilihan langkahnya dan berhenti paling cepat, tetapi paling mudah terjebak pada *local maximum* dan *plateau*.

2.1.2 *Hill Climbing with Sideways Move*

Varian ini sama dengan *steepest ascent*, tetapi juga mengizinkan perpindahan ke *neighbor* yang nilainya sama dengan *state* saat ini (*sideways move*). Dengan cara ini algoritma dapat menyeberangi *plateau* dan *shoulder*. Agar tidak berputar tanpa akhir pada *plateau* yang tidak memiliki jalan keluar, jumlah *sideways move* dibatasi oleh parameter *maximum sideways move*. Pencarian dihentikan ketika jumlah *sideways move* mencapai batas tersebut.

2.1.3 *Stochastic Hill Climbing*

Algoritma memilih satu *neighbor* secara acak pada setiap iterasi dan berpindah ke *neighbor* tersebut hanya jika nilainya lebih baik. Varian ini tidak perlu mengevaluasi seluruh *neighbor* dalam satu iterasi sehingga setiap iterasinya lebih murah, dan sifat acaknya membuat jalur pencarian lebih bervariasi. Karena tidak ada kondisi "tidak ada *neighbor* yang lebih baik" yang dapat dipastikan dari satu sampel, pencarian dihentikan setelah jumlah iterasi maksimum tercapai.

2.1.4 *Random Restart Hill Climbing*

Algoritma menjalankan *steepest ascent hill climbing* berulang kali, setiap kali dimulai dari *state* awal acak yang baru, dan menyimpan hasil terbaik dari seluruh percobaan. Peluang menemukan *global optimum* meningkat seiring bertambahnya jumlah *restart*, karena setiap *restart* mengeksplorasi daerah ruang pencarian yang berbeda. Pencarian dihentikan ketika jumlah *restart* mencapai *maximum restart*. Kelemahannya adalah biaya komputasi yang bertambah sebanding dengan jumlah *restart*.

2.2 Algoritma Simulated Annealing

2.3 Genetic Algorithm

Bab 3

Implementasi Algoritma

3.1 Representasi State

3.2 *Constraint* Permasalahan

3.3 *Succesor Function*

3.4 *Objective Function*

3.5 Formulasi *Initial State*

3.6 Implementasi Hill Climbing

Seluruh varian *hill climbing* diimplementasikan pada berkas `src/algorithms/hill_climbing/algorithm.py` di atas representasi *state*, validator *constraint*, dan fungsi *neighbor* yang sama dengan algoritma lainnya (`State`, `validate_state`, dan `random_neighbor`). Dengan demikian seluruh algoritma menerima masukan dan mengembalikan keluaran yang seragam, yaitu objek `SearchResult` yang memuat *state* awal, *state* akhir, nilai *objective function* akhir, riwayat nilai per iterasi, durasi pencarian, dan metrik khusus algoritma.

3.6.1 Pembangkitan *Neighbor* dan Heuristik

Ruang *neighbor* pada permasalahan ini sangat besar karena koordinat paket berupa bilangan bulat, sehingga seluruh *neighbor* tidak dapat dienumerasi pada setiap iterasi. Sebagai gantinya, setiap *neighbor* dibangkitkan oleh `random_neighbor`, yang secara acak memilih salah satu dari tiga operasi (menukar dua paket, memindahkan paket ke koordinat lain, atau merotasi paket 90°) dan hanya mengembalikan *state* yang lolos validasi *constraint*.

Heuristik yang kami gunakan adalah *steepest* berbasis sampel: pada setiap iterasi dibangkitkan `neighbors_per_iteration` (bawaan 30) *neighbor* valid, kemudian *neighbor* dengan nilai terbaik dianggap sebagai *neighbor* terbaik. Konsekuensinya, varian *steepest* dan *sideways* tidak menjamin menemukan *neighbor* terbaik yang sesungguhnya, tetapi peluangnya naik seiring bertambahnya ukuran sampel. Varian *stochastic* memakai tepat satu *neighbor* per iterasi.

3.6.2 Parameter dan Kondisi Berhenti

- *Steepest*: berhenti ketika *neighbor* terbaik dari sampel tidak lebih baik daripada *state* saat ini.
- *Sideways*: sama seperti *steepest*, tetapi *neighbor* bernilai sama diterima selama jumlah *sideways move* belum mencapai `max_sideways`.
- *Stochastic*: berhenti setelah `max_iterations` iterasi. Iterasi yang *neighbor*-nya ditolak tetap dihitung.
- *Random restart*: menjalankan *steepest* dari *state* awal yang diberikan, lalu dari `max_restarts` *state* acak baru, sehingga total terdapat `max_restarts` + 1 pendakian. Hasil terbaik dari seluruh pendakian dikembalikan.

Parameter	Bawaan	Keterangan
`variant`	`steepest`	`steepest`, `sideways`, `stochastic`, atau `random_restart`
`neighbors_per_iteration`	30	Jumlah *neighbor* yang disampel per iterasi (tidak dipakai *stochastic*)
`max_iterations`	1000	Batas iterasi; menjadi kondisi berhenti pada *stochastic* dan batas pengaman pada varian lain
`max_sideways`	100	Batas *sideways move* (varian *sideways*)
`max_restarts`	10	Jumlah *restart* setelah percobaan pertama (varian *random restart*)
`seed`	—	Seed pembangkit acak agar eksperimen dapat diulang

Tabel 3.1: Parameter algoritma *hill climbing*

3.6.3 Metrik yang Dicatat

Setiap *run* mengembalikan `HillClimbingMetrics` yang berisi varian yang dipakai, banyak iterasi hingga berhenti, jumlah dan batas *sideways move*, jumlah dan batas *restart*, serta banyak iterasi pada setiap *restart*. Riwayat nilai *objective function* dicatat pada setiap iterasi (elemen pertama adalah nilai *state* awal) dan digunakan untuk membuat plot nilai *objective function* terhadap banyak iterasi. Durasi pencarian diukur dengan `perf_counter`.

3.6.4 *Source Code*

Fungsi `_climb` menjalankan satu pendakian untuk varian *steepest*, *sideways*, atau *stochastic*. Fungsi `hill_climbing` memvalidasi parameter, memilih varian, menangani pengulangan pada *random restart*, lalu menyusun `SearchResult`.

Listing 3.1: Fungsi bantu dan `_climb` pada `algorithm.py`

```
from math import isfinite
from random import Random
from time import perf_counter

from src.contracts import ObjectiveFunction, resolve_objective
from src.model import State
from src.neighbors import random_initial_state, random_neighbor
from src.results import HillClimbingMetrics, ObjectiveValue, SearchResult

VARIANTS = ("steepest", "sideways", "stochastic", "random_restart")


def _check_int(value, name: str, minimum: int) -> None:
    if type(value) is not int or value < minimum:
        raise ValueError(f"{name} must be an integer >= {minimum}")


def _score(objective: ObjectiveFunction[State], state: State) -> ObjectiveValue:
    score = objective(state)
    if isinstance(score, bool) or not isinstance(score, (int, float)) or not isfinite(score):
        raise ValueError("objective_fn must return finite integer or float values")
    return score


def _climb(
    start: State,
    objective: ObjectiveFunction[State],
    rng: Random,
    *,
    variant: str,
    neighbors_per_iteration: int,
    max_iterations: int,
    max_sideways: int,
) -> tuple[State, ObjectiveValue, list[ObjectiveValue], int, int]:
    """One climb. Returns (state, score, history, iterations, sideways_moves)."
```

```
␣␣␣␣history[0]␣is␣the␣start␣score;␣one␣entry␣is␣appended␣per␣iteration.
␣␣␣␣"""
    current, current_score = start, _score(objective, start)
    history = [current_score]
    iterations = sideways = 0

    while iterations < max_iterations:
        if variant == "stochastic":
            candidates = [random_neighbor(current, rng)]
        else:
            candidates = [random_neighbor(current, rng) for _ in range(neighbors_per_iteration)]
        scored = [(_score(objective, c), c) for c in candidates]
        best_score, best = max(scored, key=lambda pair: pair[0])

        if best_score > current_score:
            current, current_score = best, best_score
        elif variant == "sideways" and best_score == current_score and sideways < max_sideways:
            current = best
            sideways += 1
        elif variant == "stochastic":
            iterations += 1  # a rejected random neighbor still costs an iteration
            history.append(current_score)
            continue
        else:
            break  # local optimum (or sideways budget exhausted)
        iterations += 1
        history.append(current_score)

    return current, current_score, history, iterations, sideways
```

Listing 3.2: Fungsi `hill_climbing` pada `algorithm.py`

```
def hill_climbing(
    initial_state: State,
    *,
    seed: int,
    objective_fn: ObjectiveFunction[State] | None = None,
    variant: str = "steepest",
    neighbors_per_iteration: int = 30,
    max_iterations: int = 1000,
    max_sideways: int = 100,
    max_restarts: int = 10,
    objective_name: str | None = None,
    run_index: int = 1,
) -> SearchResult[State]:
    """Run␣one␣hill-climbing␣variant␣and␣return␣a␣shared-contract␣SearchResult."""
    if variant not in VARIANTS:
        raise ValueError(f"variant␣must␣be␣one␣of␣{VARIANTS}")
    if type(seed) is not int:
        raise ValueError("seed␣must␣be␣an␣integer")
    _check_int(neighbors_per_iteration, "neighbors_per_iteration", 1)
    _check_int(max_iterations, "max_iterations", 0)
    _check_int(max_sideways, "max_sideways", 0)
    _check_int(max_restarts, "max_restarts", 0)
    _check_int(run_index, "run_index", 1)

    started_at = perf_counter()
    rng = Random(seed)
    objective = resolve_objective(initial_state, objective_fn)
    climb_kwargs = dict(
        variant="steepest" if variant == "random_restart" else variant,
        neighbors_per_iteration=neighbors_per_iteration,
        max_iterations=max_iterations,
        max_sideways=max_sideways,
    )

    if variant == "random_restart":
        # Restart 0 uses the supplied initial state; each later one a fresh random state.
        best_state, best_score = initial_state, None
        history: list[ObjectiveValue] = []
        per_restart: list[int] = []
        for restart in range(max_restarts + 1):
            start = (
                initial_state
                if restart == 0
```

```
                else random_initial_state(initial_state.packages, initial_state.truck, rng)
            )
            state, score, hist, iters, _ = _climb(start, objective, rng, **climb_kwargs)
            history.extend(hist)
            per_restart.append(iters)
            if best_score is None or score > best_score:
                best_state, best_score = state, score
        final_state, final_score = best_state, best_score
        iterations, sideways = sum(per_restart), 0
        restarts = max_restarts
    else:
        final_state, final_score, history, iterations, sideways = _climb(
            initial_state, objective, rng, **climb_kwargs
        )
        per_restart, restarts = [], 0

    parameters: dict[str, int | float | str | bool] = {
        "variant": variant,
        "max_iterations": max_iterations,
    }
    if variant != "stochastic":
        parameters["neighbors_per_iteration"] = neighbors_per_iteration
    if variant == "sideways":
        parameters["max_sideways"] = max_sideways
    if variant == "random_restart":
        parameters["max_restarts"] = max_restarts

    return SearchResult(
        algorithm=f"hill_climbing_{variant}",
        objective_name=objective_name or ("total_value" if objective_fn is None else "custom"),
        initial_state=initial_state,
        final_state=final_state,
        objective_value=final_score,
        objective_history=tuple(history),
        duration_seconds=perf_counter() - started_at,
        metrics=HillClimbingMetrics(
            variant=variant,
            iterations=iterations,
            sideways_moves=sideways,
            maximum_sideways_moves=max_sideways if variant == "sideways" else None,
            restarts=restarts,
            maximum_restarts=max_restarts if variant == "random_restart" else None,
            iterations_per_restart=tuple(per_restart),
        ),
        seed=seed,
        run_index=run_index,
        parameters=parameters,
    )
```

Listing 3.3: Fungsi pembungkus tiap varian pada `algorithm.py`

```
def steepest_ascent_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="steepest", **kw)


def sideways_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="sideways", **kw)


def stochastic_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="stochastic", **kw)


def random_restart_hill_climbing(initial_state: State, **kw) -> SearchResult[State]:
    return hill_climbing(initial_state, variant="random_restart", **kw)
```

3.6.5 Pelaksana Eksperimen dan Visualisasi

Berkas `src/experiments/hill_climbing.py` menjalankan setiap varian sebanyak tiga kali dengan tiga *seed* berbeda. *State* awal setiap *run* dibangkitkan secara acak dari *seed* tersebut. Untuk setiap *run*, program menyimpan berkas `result.json`, plot nilai *objective function* terhadap iterasi, dan visualisasi tiga dimensi *state* awal dan akhir. Ringkasan lintas *run* (rata-rata dan simpangan baku nilai akhir, durasi, dan iterasi)

disimpan pada `summary.json` dan `summary.png`. Program dijalankan melalui `src/experiments/run_hill_climbing.py`.

3.7 Implementasi Simullated Annealing

3.8 Implementasi Genetic Algorithm

Bab 4

Hasil Eksperimen dan Analisis

4.1 Metodologi Eksperimen

4.2 Hasil Eksperimen dan Analisis tiap Algoritma

4.2.1 Hill Climbing

4.3 Hasil Eksperimen dan Analisis tiap Algoritma

4.3.1 Hill Climbing

4.3.2 Simulated Annealing

4.3.3 Genetic Algorithm

4.4 Analisis Keseluruhan

Bab 5

Kesimpulan dan Hasil

5.1 Kesimpulan

5.2 Saran

Bibliografi

[1] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Massachusetts Institute of Technology, 2009.

[2] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.

[3] G. Nemhauser, L. Wolsey, and M. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[4] D. S. Hochbaum and A. Pathria, "Analysis of the greedy approach in problems of maximum k-coverage," *Naval Research Logistics*, vol. 45, no. 6, pp. 615–627, 1998.

[5] R. Munir, "Algoritma greedy," Lecture Notes, Institut Teknologi Bandung, 2025, accessed: 12-Feb-2026.

[6] M. Woodford, *Interest and Prices: Foundations of a Theory of Monetary Policy*. Princeton: Princeton University Press, 2003.

[7] L. E. O. Svensson, "Inflation forecast targeting: Implementing and monitoring inflation targets," *European Economic Review*, vol. 41, no. 6, pp. 1111–1146, 1997.

[8] R. Clarida, J. Galí, and M. Gertler, "The science of monetary policy: A new keynesian perspective," *Journal of Economic Literature*, vol. 37, no. 4, pp. 1661–1707, 1999.

[9] J. Galí and T. Monacelli, "Monetary policy and exchange rate volatility in a small open economy," *Review of Economic Studies*, vol. 72, no. 3, pp. 707–734, 2005.

[10] S. M. Juhro, "Kurva phillips dan perubahan struktural di indonesia: Keberadaan, pola pembentukan ekspektasi, dan linieritas," *Buletin Ekonomi Moneter dan Perbankan*, vol. 6, no. 4, pp. 41–76, 2004.

[11] T. J. Sargent and N. Wallace, "Some unpleasant monetarist arithmetic," *Federal Reserve Bank of Minneapolis Quarterly Review*, vol. 5, no. 3, pp. 1–17, 1981.

[12] E. M. Leeper, "Equilibria under 'active' and 'passive' monetary and fiscal policies," *Journal of Monetary Economics*, vol. 27, no. 1, pp. 129–147, 1991.